



Universidad autónoma de baja California

Ingeniería en computación

Sistemas embebidos

Reporte de laboratorio practica 5: I2C

Evangelina Lara Camacho

GARCIA CHAVEZ ERICK – 01275863

LERMA GARCIA ERICK JOVAN – 01281695

Viernes 17 de octubre del 2025

Parte práctica:

Explicación **ESP MASTER**:

tenemos una función base que se encarga de configurar el protocolo de I2C de una manera genérica, la función va a recibir que puerto I2C desea configurar, los pines para la comunicación de datos y de reloj y la velocidad con la que se estará comunicando.

```
1
2 void i2c_master_init(i2c_port_t num_i2c, gpio_num_t pin_sda, gpio_num_t pin_scl, uint32_t speed){
3
4     i2c_config_t conf_master = {
5         .mode = I2C_MODE_MASTER,
6         .sda_io_num = pin_sda,
7         .scl_io_num = pin_scl,
8         .sda_pullup_en = GPIO_PULLUP_ENABLE,
9         .scl_pullup_en = GPIO_PULLUP_ENABLE,
10        .master.clk_speed = speed,
11    };
12
13    esp_err_t ret = i2c_param_config(num_i2c, &conf_master);
14    if (ret != ESP_OK) {
15        ESP_LOGE(TAG, "I2C master config fallido: %s", esp_err_to_name(ret));
16    }
17
18    ret = i2c_driver_install(num_i2c, conf_master.mode, 0, 0, 0);
19    if (ret != ESP_OK) {
20        ESP_LOGE(TAG, "I2C master-driver fallido: %s", esp_err_to_name(ret));
21    }
22
23    ESP_LOGI(TAG, "Bus I2C maestro para sensor inicializado correctamente");
24 }
```

Como la comunicación tendrá un “protocolo de comunicación” definimos una macros que nos ayudaran a configurar esto y pasarlo por el bus de datos

```
1 #define I2C_REQUEST_HEADER 0x1F
2 #define I2C_REQUEST_CMD 0x28
3 #define I2C_RESPONSE_HEADER 0x2F
4 #define I2C_RESPONSE_CMD 0x28
```

Esto lo que hace es que 0x1F “inicio de mensaje”, 0x28 leer temperatura del sensor.

El uso de una estructura para guardar la información recibida del **ESP SLAVE**, esto lo hago porque el sensor LM75ab nos regresa un dato de 16 bits con signo, esta estructura me simplifica para trabajar con cada parte por separado y mas por la parte decimal, el sensor tiene una resolución de 0.5 C, pero como se sabe que los microcontroladores las operaciones de punto flotante son algo pesadas para estos, escalo el valor en su parte decimal para calcular el valor real que se esta trayendo desde el sensor, además con la estructura es mucho más fácil el mostrar el resultado por UART

```
1  typedef struct{
2      int8_t integer;
3      uint8_t decimal;
4  }temperature_t;
5
6
```

Función en donde el master solicita una lectura, esta función lo que hace es establecer la comunicación, indica a que dispositivo quiere establecer una comunicación, usamos un arreglo que tiene el protocolo para indicar que queremos leer un dato. Este se envia crudo, es una función de tipo booleano para saber si se pudo comunicar o no.

```
1
2  bool request_master(){
3
4      esp_err_t ret;
5
6      uint8_t request[2]= {I2C_REQUEST_HEADER, I2C_REQUEST_CMD};
7
8
9
10     i2c_cmd_handle_t cmd_handle = i2c_cmd_link_create();
11
12     i2c_master_start(cmd_handle);
13
14     i2c_master_write_byte(cmd_handle, (I2C_ESP_SLAVE << 1) | (I2C_MASTER_WRITE), ACK_CHECK_EN);
15     i2c_master_write(cmd_handle, request, sizeof(request), ACK_CHECK_EN);
16     i2c_master_stop(cmd_handle);
17
18
19     ret = i2c_master_cmd_begin(I2C_MASTER_PORT, cmd_handle, pdMS_TO_TICKS(TIME_MS));
20     i2c_cmd_link_delete(cmd_handle);
21
22     if(ret != ESP_OK){
23
24         ESP_LOGE(TAG, "Fallo al enviar la solicitud: %s", esp_err_to_name(ret));
25         return false;
26     }
27     else{
28         return true;
29     }
30
31     return false;
32 }
```

Ahora esta función es la que recibe los datos del **ESP SLAVE** indicamos que queremos leer, y guardamos lo que mando el **ESP SLAVE** en el arreglo de **response** antes de leer o guardar el dato dentro del apuntador debemos de verificar que lo que nos envió está en el formato correcto, este formato está en los primeros 2 bytes de la respuesta que es el HEADER y el CMD del

Slave -> master si no es correcto al protocolo se destaca los datos leídos.

En otro caso se pasa a guardar los datos, la parte entera que es el byte MSB donde el MSB repente el signo y los siguientes 7 bits es la parte entera, el bit 7 representa la parte de fracción con una resolución de 0.5 c, y los bits de 6 – 0 no se usan, pero la respuesta sigue siendo de 16 bits. Por lo que la parte de fracción lo multiplico por 5 para sacar su parte incrementada

```
1
2
3 bool requets_temperature(temperature_t *temperature){
4
5
6     uint8_t response[4]={0};
7
8     esp_err_t ret;
9
10    i2c_cmd_handle_t cmd_handle = i2c_cmd_link_create();
11    i2c_master_start(cmd_handle);
12    i2c_master_write_byte(cmd_handle, (I2C_ESP_SLAVE << 1) | I2C_MASTER_READ, ACK_CHECK_EN);
13    i2c_master_read(cmd_handle, response, sizeof(response), I2C_MASTER_LAST_NACK);
14
15    i2c_master_stop(cmd_handle);
16
17    ret = i2c_master_cmd_begin(I2C_MASTER_PORT, cmd_handle, pdMS_TO_TICKS(TIME_MS));
18    i2c_cmd_link_delete(cmd_handle);
19
20    if(ret != ESP_OK){
21        ESP_LOGE(TAG, "Fallo al enviar la solicitud: %s", esp_err_to_name(ret));
22        return false;
23    }
24
25    //validamos la respuesta
26
27    if(response[0] != I2C_RESPONSE_HEADER || response[1] != I2C_RESPONSE_CMD){
28        const char *mess = "formato de respuesta invalido\n";
29        uart_write_bytes(UART_USE, mess, strlen(mess));
30        return false;
31    }
32
33    int16_t temp = (response[2] << 8) | response[3];
34
35    int8_t temp_integer = (int8_t)response[2]; //parte entera con signo
36    uint8_t temp_decimal = (response[3] & 0x80) >>7;
37
38    temperature->integer = temp_integer;
39    temperature->decimal = temp_decimal * 5;
40
41    return true;
42 }
43
```

Nuestra tarea principal lo que hace es llamar a las funciones para establecer y leer del buffer y lo que hace es que va a estar leyendo siempre y cuando se pueda establecer una comunicación o que el formato sea el correcto si no es correcto lo intentara máximo 3 veces en otro caso ya no leerá y cerrara la comunicación, pero siempre y cuando pase con éxito desplegara por UART el valor leído que está en la variable de tipo temperature_t, si no, sale del primero while y pasa al siguiente en donde se quedara bloqueado

```
1 void master_task(void *parms){
2     ESP_LOGI(TAG, "I2C MASTER inicializado");
3
4
5     while(1){
6
7         bool communication_ok = false;
8         temperature_t received_temp;
9         for(int i = 0 ; i< MAX_ATTEMPTS; i++){
10
11             ESP_LOGI(TAG, "intento %d de comunicacion ", i+1);
12
13
14
15
16             //si la comunicacion del master con el slave es correcta entonces
17             if(request_master()){
18                 //digamos que si logo realizar la lectrua
19
20                 if(requets_temperature(&received_temp)){
21
22                     //si recibio la temperatura en el formato correcto
23                     char buff_int[5]; //para almacenar hasta 5 bytes de informacion, los justos para repsetar -128\0 hasta 127\0
24
25                     sprintf(buff_int, "%d",received_temp.integer);
26                     uart_write_bytes(UART_USE,buff_int, strlen(buff_int));
27
28                     uart_write_bytes(UART_USE,".", 1);
29
30                     char buff_dec[5];
31                     sprintf(buff_dec, "%d",received_temp.decimal);
32                     uart_write_bytes(UART_USE, buff_dec, strlen(buff_dec));
33                     communication_ok = true;
34                     uart_write_bytes(UART_USE,"\n", 1);
35                     break;
36                 }
37             }else{
38                 ESP_LOGE(TAG, "error en la recepcion");
39             }
40         }else{
41
42             ESP_LOGE(TAG, "intento de comunicacion # <%d> fallido...", i+1);
43
44             continue;
45         }
46     }
47 }
48 if (!communication_ok) {
49     const char *mess = "Comunicacion terminada, el periferico no responde\n";
50     uart_write_bytes(UART_USE, mess, strlen(mess));
51     break;
52 }
53
54 vTaskDelay(2000 / portTICK_PERIOD_MS);
55 }
56 while(1){
57
58     vTaskDelay(1000 / portTICK_PERIOD_MS);
59 }
60 }
```

Uart MASTER

Solo iniciamos el uart, no ocupamos tarea para cachar los datos de RX porque solo va a mostrar solo utilizara TX:

```
esp_err_t init_uart(uart_port_t num_uart, int pin_tx, int pin_rx, uart_word_length_t data, uart_parity_t bits_parity, uart_stop_bits_t stop_bits_f){

    ESP_LOGI(TAG, "CONFIGURANDO UART");

    uart_config_t uart_config = {
        .baud_rate = 115200,
        .data_bits = data,
        .parity = bits_parity,
        .stop_bits = stop_bits_f,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };

    if(num_uart >= MAX_UART){
        //error al querer configurar un UART fura del rango
        return ESP_FAIL;
    }

    else{
        current_uart = num_uart;
        //el numero del uart entra bien
        ESP_ERROR_CHECK(uart_param_config(num_uart, &uart_config));
        ESP_ERROR_CHECK(uart_driver_install(num_uart, BUFF*2, BUFF*2, 20, &event_uart, 0));
        ESP_ERROR_CHECK(uart_set_pin(num_uart, pin_tx, pin_rx, UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));

        uart_flush(num_uart);

        return ESP_OK; //el uart se configuro correctamente
    }

    return ESP_FAIL; //EN CASO DE cualquier error retornar false
}
```

ESP SLAVE:

Tenemos la función de iniciar como master de la misma manera, es la misma función.

```
void i2c_master_init(i2c_port_num_t num_i2c, gpio_num_t pin_sda, gpio_num_t pin_scl, uint32_t speed) {

    i2c_config_t conf_master = {
        .mode = I2C_MODE_MASTER,
        .sda_io_num = pin_sda,
        .scl_io_num = pin_scl,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .master.clk_speed = speed,
    };

    esp_err_t ret = i2c_param_config(num_i2c, &conf_master);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "I2C master param config failed: %s", esp_err_to_name(ret));
    }

    ret = i2c_driver_install(num_i2c, conf_master.mode, 0, 0, 0);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "I2C master driver install failed: %s", esp_err_to_name(ret));
    }

    ESP_LOGI(TAG, "Bus I2C maestro para sensor inicializado correctamente");
}
```

Este tiene la función para establecer un puerto i2c como esclavo, la cual la función es prácticamente igual solo que en modo establecemos que es para esclavo, así como cuando se instalan los drivers: y cuál es su dirección de esclavo

```
void i2c_slave_init(i2c_port_num_t num_i2c, gpio_num_t pin_sda, gpio_num_t pin_scl, uint8_t slave_addr){

    i2c_config_t conf_slave = {
        .mode = I2C_MODE_SLAVE,
        .sda_io_num = pin_sda,
        .scl_io_num = pin_scl,
        .sda_pullup_en = GPIO_PULLUP_ENABLE,
        .scl_pullup_en = GPIO_PULLUP_ENABLE,
        .slave.addr_10bit_en = 0,
        .slave.slave_addr = slave_addr,
    };

    esp_err_t ret = i2c_param_config(num_i2c, &conf_slave);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "I2C slave param config failed: %s", esp_err_to_name(ret));
    }

    ret = i2c_driver_install(num_i2c, conf_slave.mode, I2C_SLAVE_RX_BUF_LEN, I2C_SLAVE_TX_BUF_LEN, 0);
    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "I2C slave driver install failed: %s", esp_err_to_name(ret));
    }

    ESP_LOGI(TAG, "I2C esclavo inicializado en dirección 0x%02X", slave_addr);
}
```

Tenemos los mismos valores como protocolo:

```
//protocolo de comunicacion
#define I2C_REQUEST_HEADER 0x1F
#define I2C_REQUEST_CMD 0x28
#define I2C_RESPONSE_HEADER 0x2F
#define I2C_RESPONSE_CMD 0x28
```

La función **read_sens_tmp** será la función en donde el ESP se vuelve MASTER y le solicita al sensor los datos que esta leyendo.

Tenemos estas 2 variables dentro de la función en donde **reg_addr = 0x00** es el registro del sensor que indica que queremos leer la temperatura y **data** como el sensor devuelve el valor que esta leyendo en un frame de 16 bits por lo que en la estructura lo hacemos de 8 bits esto porque solo los primeros 11 bits realmente nos dicen la lectura de la, por lo que los ponemos en el primer byte mas significativo y la otra parte, los 5 restantes siempre son 0, no se

utilizan en la lectura, entonces esa variable es la que utilizamos para guardar lo que leyó del sensor:

```
uint8_t reg_addr = 0x00;
uint8_t data[2] = {0};
```

Por lo que esta función realiza la petición de quiero leer en x dirección y después realiza la lectura como tal, como se ve en las siguientes líneas que indica que quiere leer la dirección del sensor que por defecto es la **0x48**

```
i2c_master_write_byte(cmd, (LM75_SEN_ADDR << 1) | I2C_MASTER_WRITE, true);
i2c_master_write_byte(cmd, reg_addr, true);
```

Ahora leemos el buffer y lo guardamos en el arreglo:

```
i2c_master_write_byte(cmd, (LM75_SEN_ADDR << 1) | I2C_MASTER_READ, true);
// Leer los 2 bytes de datos
i2c_master_read(cmd, data, 2, I2C_MASTER_LAST_NACK);
```

```
bool read_sens_tmp(int16_t *tmp) {
    uint8_t reg_addr = 0x00;
    uint8_t data[2] = {0};

    i2c_cmd_handle_t cmd = i2c_cmd_link_create();
    i2c_master_start(cmd);
    // Escribir la dirección del sensor y el registro que queremos leer
    i2c_master_write_byte(cmd, (LM75_SEN_ADDR << 1) | I2C_MASTER_WRITE, true);
    i2c_master_write_byte(cmd, reg_addr, true);
    // Reiniciar para leer
    i2c_master_start(cmd);
    // Escribir la dirección del sensor y solicitar lectura
    i2c_master_write_byte(cmd, (LM75_SEN_ADDR << 1) | I2C_MASTER_READ, true);
    // Leer los 2 bytes de datos
    i2c_master_read(cmd, data, 2, I2C_MASTER_LAST_NACK);
    i2c_master_stop(cmd);

    esp_err_t ret = i2c_master_cmd_begin(I2C_SEN_PORT, cmd, pdMS_TO_TICKS(1000));
    i2c_cmd_link_delete(cmd);

    if (ret != ESP_OK) {
        ESP_LOGE(TAG, "error en la lectura al sensor: %s", esp_err_to_name(ret));
        return false;
    }

    *tmp = (data[0] << 8) | data[1];
    return true;
}
```


La tarea de Slave que es la que responde al ESP master,

Como esta función es la que le manda la información al master necesitamos verificar que el protocolo sea el correcto, tenemos 2 arreglos, 1 que verifica que la comunicación sea correcta y otra que es donde se enviara el dato, los datos se envían “crudos” porque es en el master en donde se tratan para poder imprimirlos por UART.

```
uint8_t request[2] = {0};  
uint8_t response[4] = {0};
```

Tenemos esta primera parte estamos leyendo el buffer y verificando que sea a nosotros a quien el master esta llamando, para esto verificamos que en la respuesta el protocolo sea el correcto si es así puede seguir en la operación si no, manda un error al leer el buffer ya que los datos pueden venir corruptos si es que es nuestra dirección a quien se comunico

```
int size_read = i2c_slave_read_buffer(I2C_PORT_SLAVE, request, 2, portMAX_DELAY);  
  
if (size_read != 2) {  
    ESP_LOGE(TAG, "Error al leer el buffer del master");  
    continue;  
}  
else{  
    if(request[0] == I2C_REQUEST_HEADER && request[1] == I2C_REQUEST_CMD){
```

En la siguiente parte, mandamos a llamar a la función antes descrita y construimos nuestra comunicación, la respuesta es de 4 bytes porque son 2 de la cabecera de la comunicación y 2 de los datos y estos el master debe de verificar que la cabecera que son los primeros 2 bytes sean los correctos en la comunicación, y los manda por el buffer al master:

```
int16_t raw_tmp;  
  
if(read_sens_tmp(&raw_tmp)){  
    response[0] = I2C_RESPONSE_HEADER;  
    response[1] = I2C_RESPONSE_CMD;  
    response[2] = (raw_tmp >> 8) & 0xFF;  
    response[3] = raw_tmp & 0xFF;  
  
    //enviamos al ESP  
  
    i2c_slave_write_buffer(I2C_PORT_SLAVE, response, 4, portMAX_DELAY);
```

```

void slave_task(void *parms){

    ESP_LOGI(TAG , "TASK SLAVE INIT");

    uint8_t request[2] = {0};
    uint8_t response[4] = {0};

    while(1){

        int size_read = i2c_slave_read_buffer(I2C_PORT_SLAVE, request, 2, portMAX_DELAY);

        if (size_read != 2) {
            ESP_LOGE(TAG, "Error al leer el buffer del master");
            continue;
        }
        else{
            if(request[0] == I2C_REQUEST_HEADER && request[1] == I2C_REQUEST_CMD){
                ESP_LOGI(TAG, "solicitud recibida correctamente");

                int16_t raw_tmp;

                if(read_sens_tmp(&raw_tmp)){
                    response[0] = I2C_RESPONSE_HEADER;
                    response[1] = I2C_RESPONSE_CMD;
                    response[2] = (raw_tmp >> 8) & 0xFF;
                    response[3] = raw_tmp & 0xFF;

                    //enviamos al ESP

                    i2c_slave_write_buffer(I2C_PORT_SLAVE, response, 4, portMAX_DELAY);
                }else{
                    ESP_LOGE(TAG, "error al leer el sensor");
                }
            }
            else{
                ESP_LOGE(TAG, "error en la recepcion");
            }
        }
    }
}

```

Respuestas:

Cunado funciona perfectamente manda la respuesta, la ltemperatura que esta leyendo acualmente:

```
28.0
I (12328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (14328) MAIN - MASTER: intento 1 de comunicacion
28.0
I (16328) MAIN - MASTER: intento 1 de comunicacion
28.0
I (18328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (20328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (22328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (24328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (26328) MAIN - MASTER: intento 1 de comunicacion
28.5
I (298) MAIN - SLAVE: TASK SLAVE INIT
I (308) main task: Returned from app_main()
I (5428) MAIN - SLAVE: solicitud recibida correctamente
I (5438) MAIN - SLAVE: solicitud recibida correctamente
I (7438) MAIN - SLAVE: solicitud recibida correctamente
I (9438) MAIN - SLAVE: solicitud recibida correctamente
I (11438) MAIN - SLAVE: solicitud recibida correctamente
I (13438) MAIN - SLAVE: solicitud recibida correctamente
I (15438) MAIN - SLAVE: solicitud recibida correctamente
I (17438) MAIN - SLAVE: solicitud recibida correctamente
I (19438) MAIN - SLAVE: solicitud recibida correctamente
I (21438) MAIN - SLAVE: solicitud recibida correctamente
I (23438) MAIN - SLAVE: solicitud recibida correctamente
I (25438) MAIN - SLAVE: solicitud recibida correctamente
I (27438) MAIN - SLAVE: solicitud recibida correctamente
I (29438) MAIN - SLAVE: solicitud recibida correctamente
I (31438) MAIN - SLAVE: solicitud recibida correctamente
```

Que pasaría si un cable ya sea del SDA o del SCL no esta conectados, no se podrá comunicar y el programa marcario que no se pudo comunicar y saldría

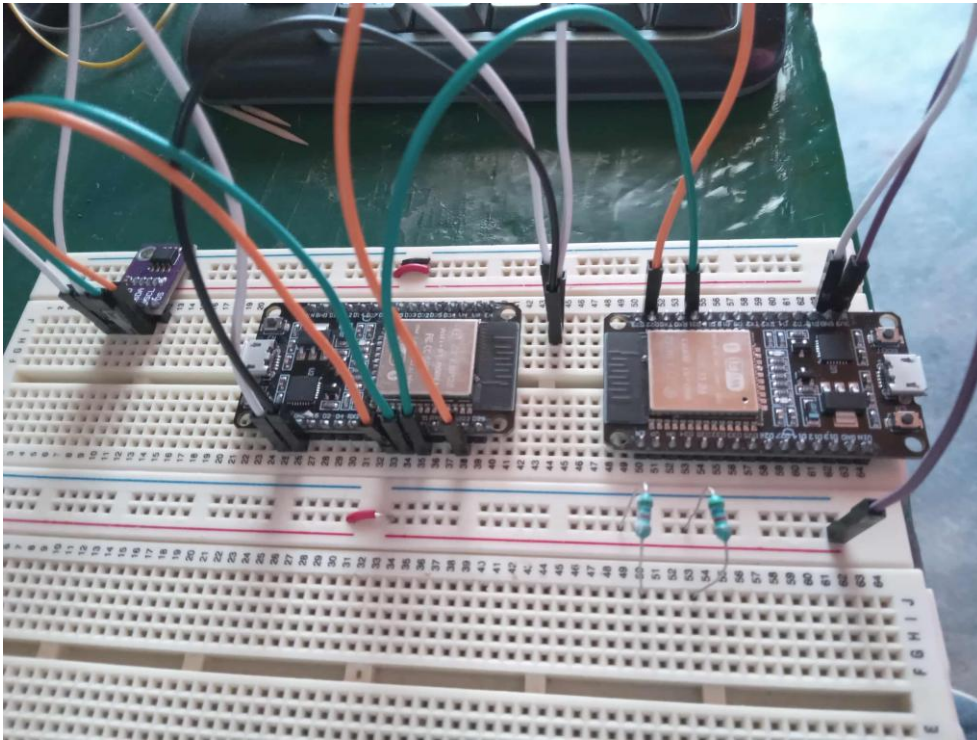
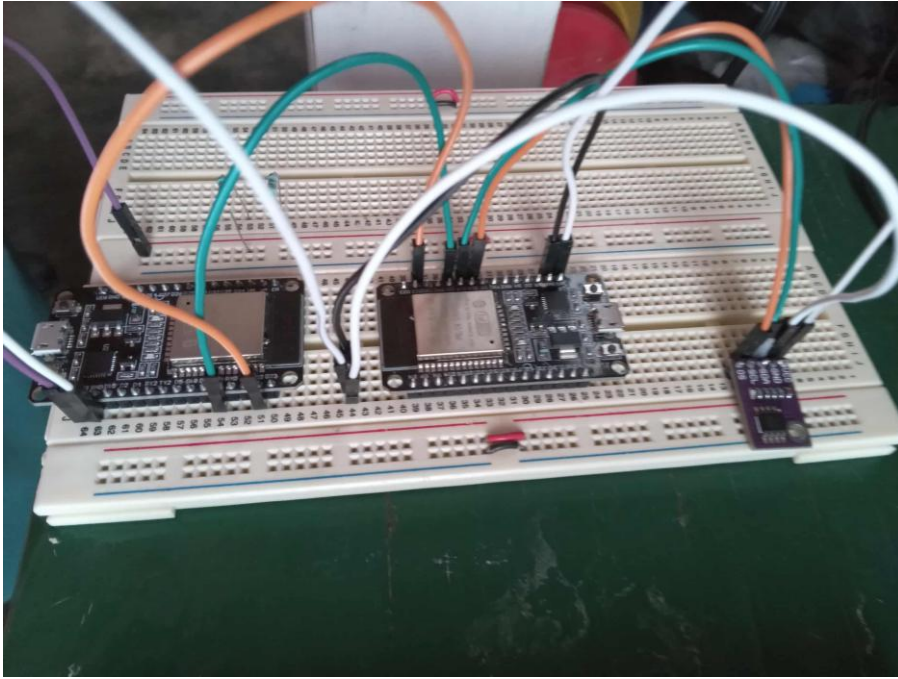
Falla entre EPS'S, si algún cable o algo esta mal entre el MASTER-SLAVE solo lo intenta 3 veces y cierra la comunicacion

```
I (308) MAIN - MASTER: I2C MASTER inicializado
I (308) MAIN - MASTER: intento 1 de comunicacion
I (308) main task: Returned from app_main()
E (308) MAIN - MASTER: Fallo al enviar la solicitud: ESP_FAIL
E (318) MAIN - MASTER: intento de comunicacion # <1> fallido...
I (328) MAIN - MASTER: intento 2 de comunicacion
E (328) MAIN - MASTER: Fallo al enviar la solicitud: ESP_FAIL
E (338) MAIN - MASTER: intento de comunicacion # <2> fallido...
I (348) MAIN - MASTER: intento 3 de comunicacion
E (348) MAIN - MASTER: Fallo al enviar la solicitud: ESP_FAIL
E (348) MAIN - MASTER: intento de comunicacion # <3> fallido...
Comunicacion terminada, el periferico no responde
I (263) spi flash: flash io: dio
W (266) spi flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the size in the binary image header.
W (278) i2c: This driver is an old driver, please migrate your application code to adapt 'driver/i2c_master.h'
I (288) main task: Started on CPU0
I (298) main task: Calling app_main()
I (298) I2C SLAVE: Bus I2C maestro para sensor inicializado correctamente
I (298) I2C SLAVE: I2C esclavo inicializado en dirección 0x40
I (298) MAIN - SLAVE: TASK SLAVE INIT
I (308) main task: Returned from app_main()
```

Fallo el sensor, lo mismo ocurrirá si falla la lectura del sensor

```
I (298) main task: Calling app_main()
I (308) I2C MASTER: Bus I2C maestro para sensor inicializado correctamente
I (308) MAIN - MASTER: I2C MASTER inicializado
I (308) MAIN - MASTER: intento 1 de comunicacion
I (308) main task: Returned from app_main()
E (318) MAIN - MASTER: error en la recepcion
I (318) MAIN - MASTER: intento 2 de comunicacion
E (328) MAIN - MASTER: error en la recepcion
I (328) MAIN - MASTER: intento 3 de comunicacion
E (338) MAIN - MASTER: error en la recepcion
formato de respuesta invalido
formato de respuesta invalido
formato de respuesta invalido
Comunicacion terminada, el periferico no responde
I (298) main task: Calling app_main()
I (298) I2C SLAVE: Bus I2C maestro para sensor inicializado correctamente
I (298) I2C SLAVE: I2C esclavo inicializado en dirección 0x40
I (298) MAIN - SLAVE: TASK SLAVE INIT
I (308) main task: Returned from app_main()
I (68228) MAIN - SLAVE: solicitud recibida correctamente
E (68228) MAIN - SLAVE: error en la lectura al sensor: ESP_FAIL
E (68228) MAIN - SLAVE: error al leer el sensor
I (68248) MAIN - SLAVE: solicitud recibida correctamente
E (68248) MAIN - SLAVE: error en la lectura al sensor: ESP_FAIL
E (68248) MAIN - SLAVE: error al leer el sensor
I (68248) MAIN - SLAVE: solicitud recibida correctamente
E (68258) MAIN - SLAVE: error en la lectura al sensor: ESP_FAIL
E (68258) MAIN - SLAVE: error al leer el sensor
```

Parte física:



Dificultades:

Erik garcia Chávez:

Una de las principales dificultades encontradas fue la inconsistencia entre la documentación de la API de I2C más reciente del ESP-IDF y la implementación real en nuestra versión de trabajo (v5.1.1). El objetivo inicial era utilizar las funciones del nuevo driver como `i2c_new_slave_device` y `i2c_slave_receive`. Sin embargo, nos topamos con errores de compilación y comportamiento inesperado, específicamente porque la documentación sugería la existencia de parámetros (como un timeout para la función de recepción del esclavo) que no estaban presentes en la declaración de la función de nuestra versión del SDK.

Erik giovan Lerma garcia:

Otra dificultad significativa se presentó en la capa física de la comunicación. Inicialmente, aun con una lógica de software aparentemente correcta, la comunicación entre los dos ESP32 era muy inestable, presentando errores esporádicos de timeout y NACK (Not Acknowledge). El problema no residía en el código, sino en la integridad de la señal del bus I2C. Se determinó que las resistencias de pull-up internas de los microcontroladores eran insuficientes para mantener un estado ALTO estable en las líneas SDA y SCL, especialmente al conectar dos placas distintas a través de cables en un protoboard. Se agregaron pull ups externos de 5.6Kohms, pero después se eliminaron en forma de experimento y funciono, entonces en ocasiones parece que hace cosas raras o nos faltó en tocar algo que no nos dimos cuenta

Conclusiones:

Erik garcia Chávez:

Este proyecto cumplió con el objetivo de familiarizarnos con el periférico I2C para la comunicación entre microcontroladores y sensores. La conclusión principal es la importancia de la **adaptabilidad y la validación del entorno de desarrollo**. Se aprendió que la documentación oficial puede no siempre corresponder exactamente a la versión específica de un SDK, y es crucial saber diagnosticar estas discrepancias. La decisión de migrar a la API de I2C antigua no fue un retroceso, sino una solución de ingeniería pragmática que nos permitió construir un sistema robusto y funcional. Esta experiencia demostró que un conocimiento profundo de las herramientas, incluyendo sus versiones y posibles limitaciones, es tan valioso como la habilidad de programar la lógica de la aplicación en sí.

Erik Jovan Lerma García:

A través del desarrollo de esta práctica, se concluyó que la implementación exitosa de un protocolo de comunicación como I2C requiere un enfoque integral que abarque tanto el software como el hardware. La depuración de errores a nivel de señal eléctrica nos enseñó que un código lógicamente perfecto puede fallar si no se considera el entorno físico. Por lo tanto, se logró no solo una comprensión teórica del protocolo I2C, sino también una apreciación práctica de su implementación, desde los voltajes en los pines hasta el manejo de datos en la tarea de la aplicación final.